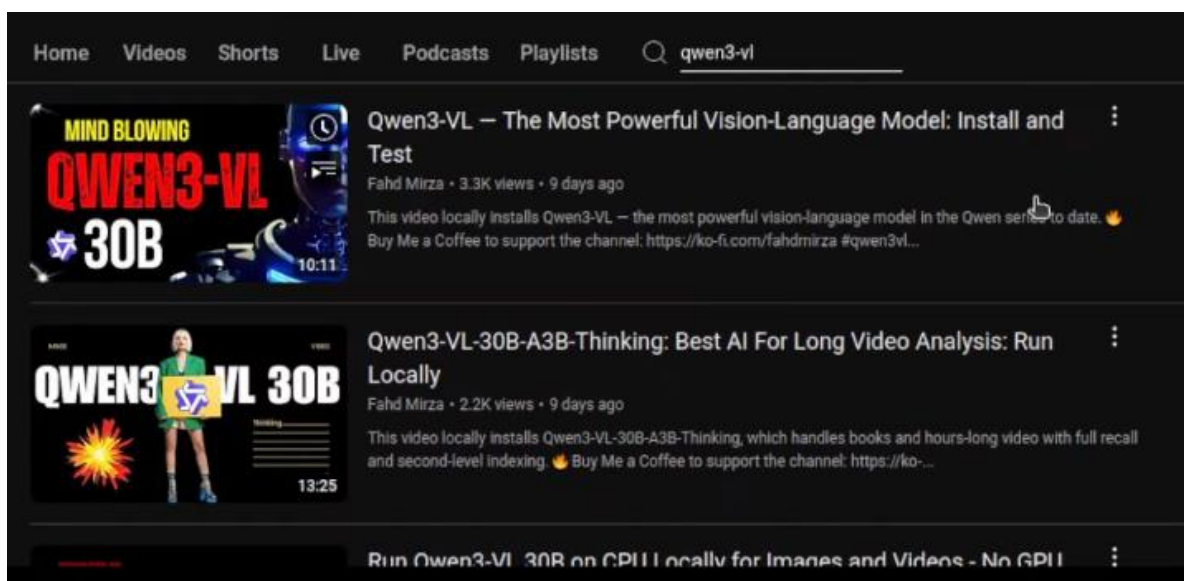
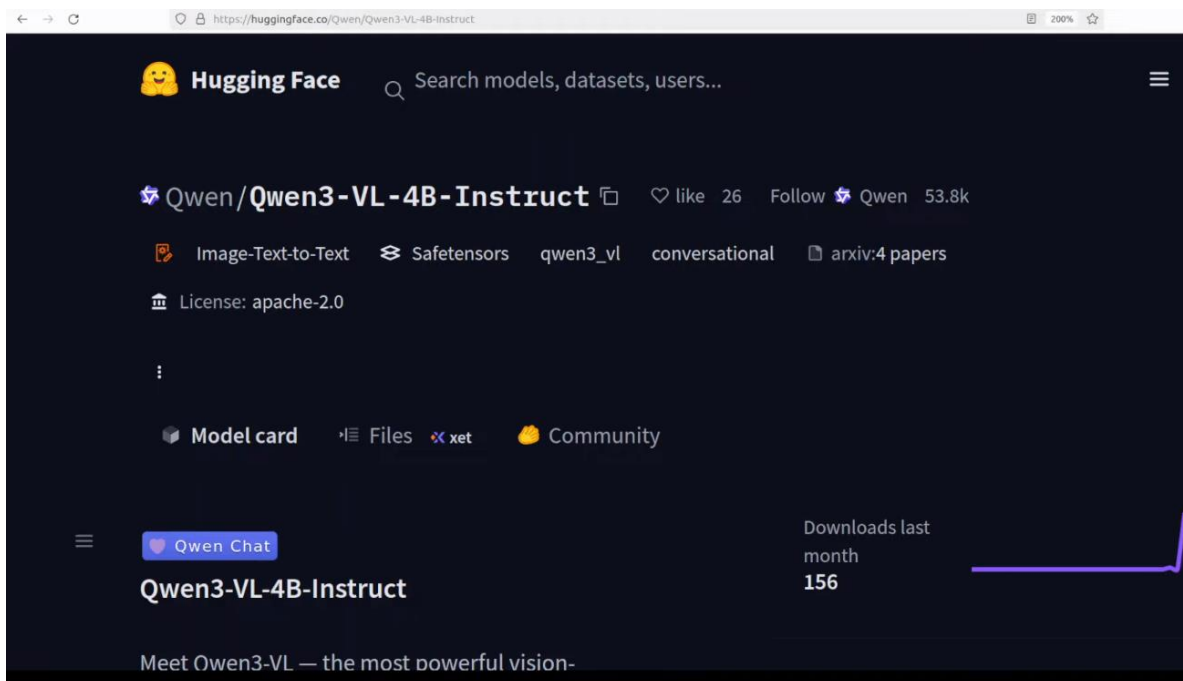


47.8K subscribers

Subscribe

 Share

This video locally installs Qwen3-VL-4B Instruct— the most powerful small vision-language model in the Qwen series to date.



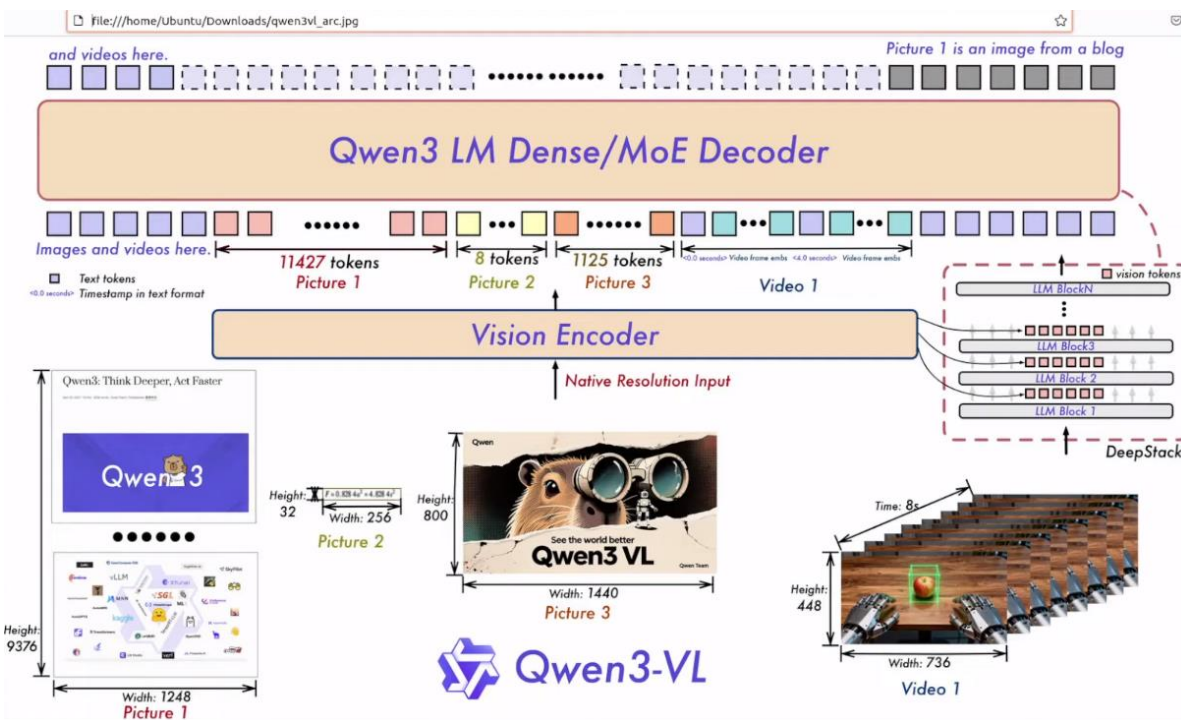
```

#
# $ conda deactivate

(ai) Ubuntu@0127-dsm2-ty6k-prxmx70026:~$ pip install torch torchvision torchaudio --index-url https
://download.pytorch.org/whl/cu121
pip install einops timm pillow
pip install git+https://github.com/huggingface/transformers
pip install git+https://github.com/huggingface/accelerate
pip install git+https://github.com/huggingface/diffusers
pip install huggingface hub ipython
pip install sentencepiece bitsandbytes protobuf decord numpy

conda install -y -c conda-forge "ffmpeg<8"
export LD_LIBRARY_PATH="$CONDA_PREFIX/lib:${LD_LIBRARY_PATH}"
pip uninstall -y torchcodec
pip install --no-cache-dir --index-url https://download.pytorch.org/whl/cu121 "torchcodec==0.1.1+cu
121"
Looking in indexes: https://download.pytorch.org/whl/cu121
Collecting torch
  Using cached https://download.pytorch.org/whl/cu121/torch-2.5.1%2Bcu121-cp311-cp311-linux_x86_64.
whl (780.5 MB)

```



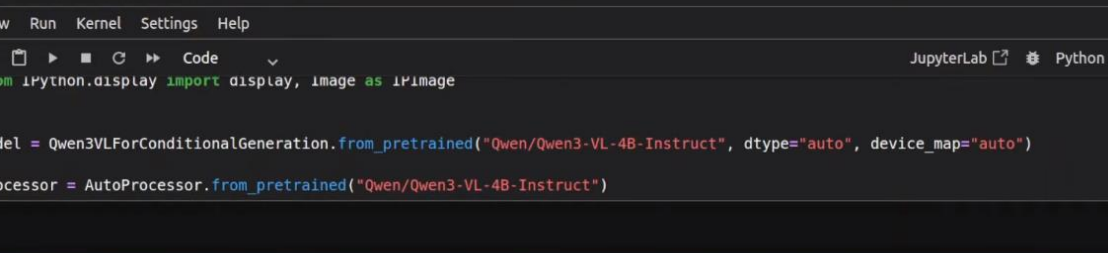
```

#
# $ conda deactivate

(ai) Ubuntu@0127-dsm2-ty6k-prxmx70026:~$ hf auth login

To log in, `huggingface_hub` requires a token generated from https://huggingface.co/settings/to
kens .
Enter your token (input will not be visible): █

```

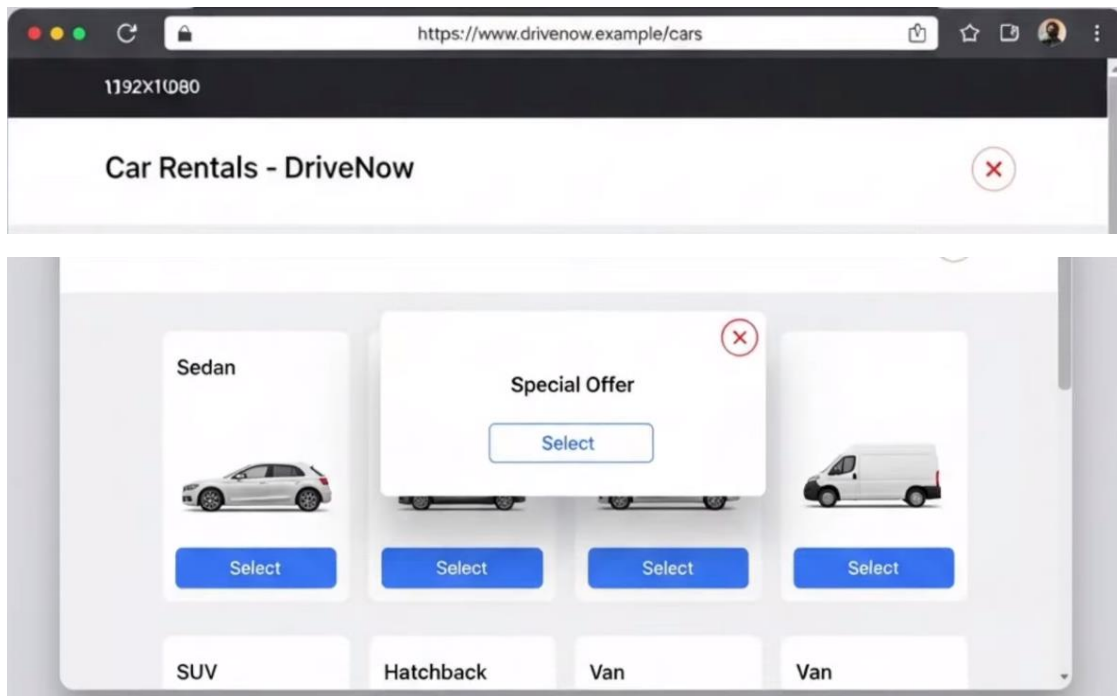
```
from IPython.display import display, Image as IPImage

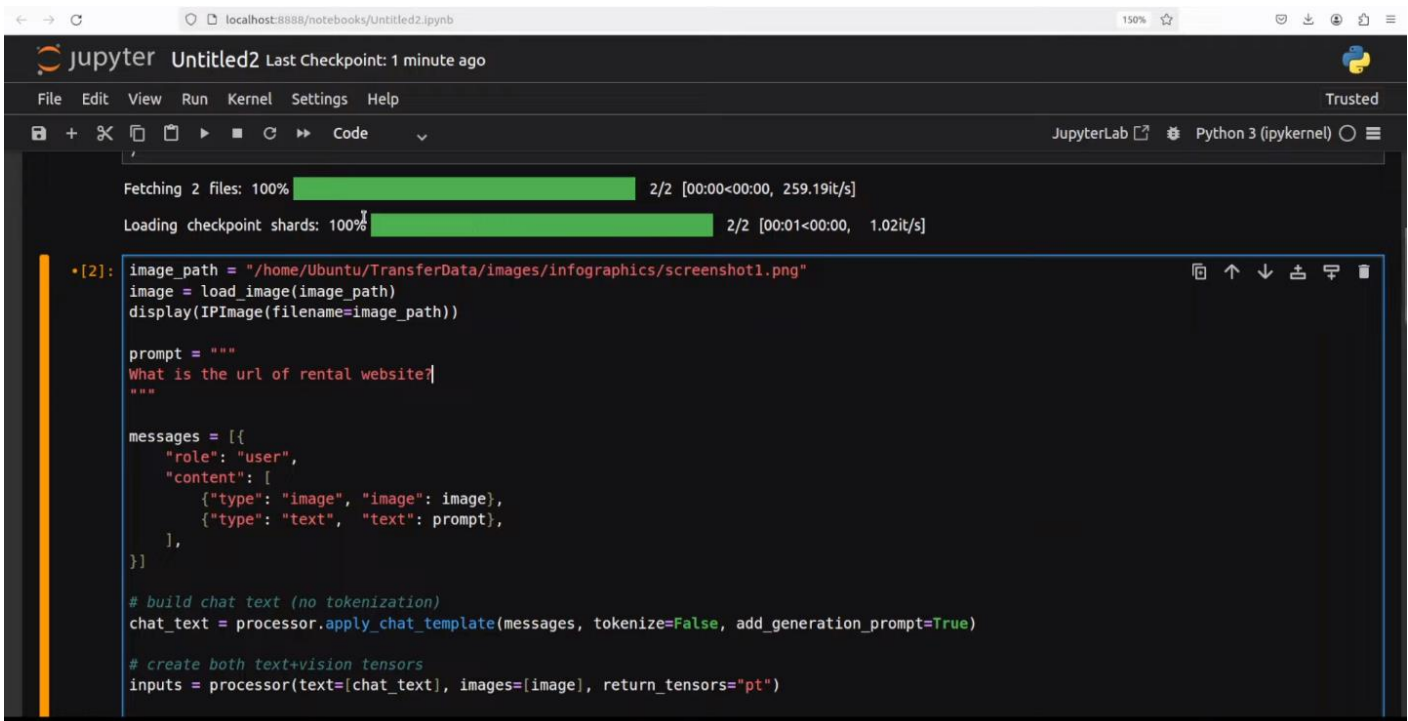
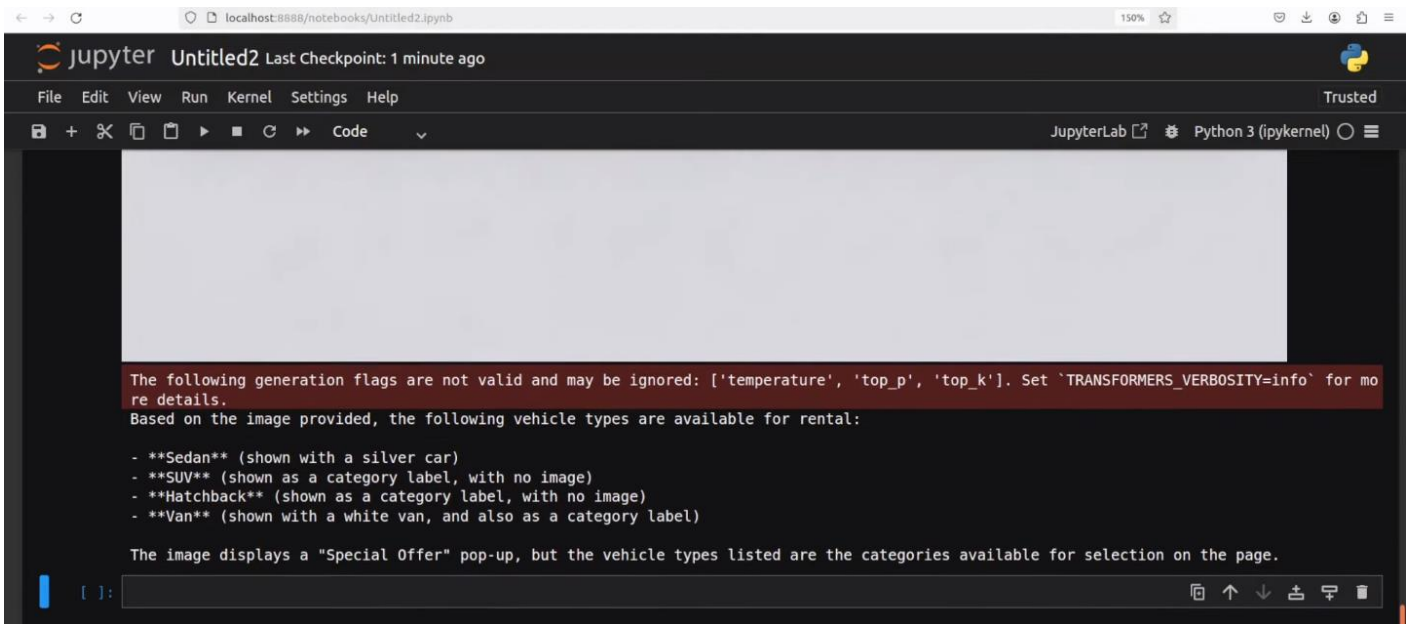
model = Qwen3VLForConditionalGeneration.from_pretrained("Qwen/Qwen3-VL-4B-Instruct", dtype="auto", device_map="auto")
processor = AutoProcessor.from_pretrained("Qwen/Qwen3-VL-4B-Instruct")

[ ]: image_path = "/home/Ubuntu/TransferData/images/infographics/screenshot1.png"
image = load_image(image_path)
display(IPImage(filename=image_path))
prompt="""
Which vehicle types are available for rental in this image?
"""

messages = [{"role": "user", "content": [{"type": "image", "image": image}, {"type": "text", "text": prompt}],}]
inputs = processor.apply_chat_template(messages, tokenize=True, add_generation_prompt=True, return_dict=True, return_tensors="pt")

generated_ids = model.generate(**inputs, max_new_tokens=128)
generated_ids_trimmed = [
    out_ids[len(in_ids):] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False)
print(output_text)
```





```
localhost:8888/notebooks/Untitled2.ipynb 150%
Jupyter Untitled2 Last Checkpoint: 1 minute ago Trusted
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)

# build chat text (no tokenization)
chat_text = processor.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)

# create both text+vision tensors
inputs = processor(text=[chat_text], images=[image], return_tensors="pt")

# ensure tensors are on the same device as the model
inputs = {k: v.to(model.device) for k, v in inputs.items()}

# generate
generated_ids = model.generate(
    **inputs,
    max_new_tokens=128,
    do_sample=False,
    pad_token_id=processor.tokenizer.eos_token_id,
)

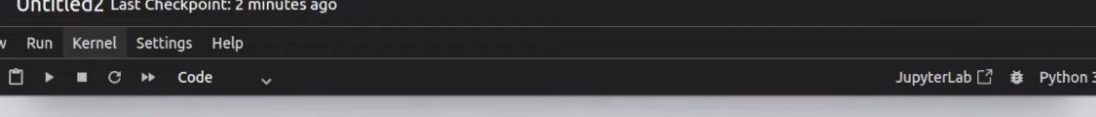
# trim prompt tokens before decoding (matches your pattern)
generated_ids_trimmed = [
    out_ids[len(in_ids):] for in_ids, out_ids in zip(inputs["input_ids"], generated_ids)
]

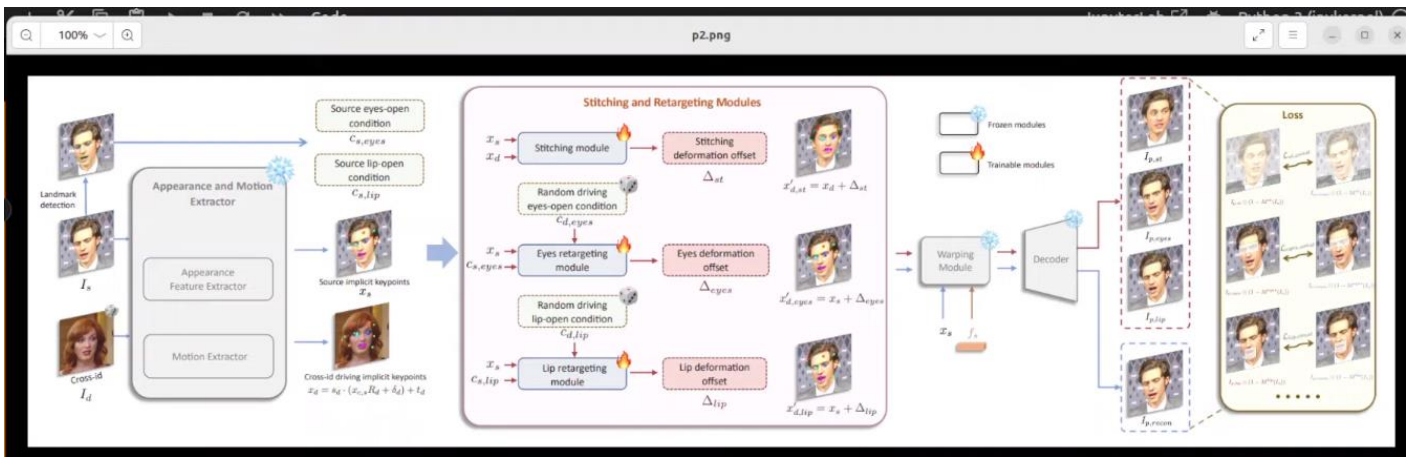
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
print(output_text[0] if isinstance(output_text, list) else output_text)
```

PID	USER	DEV	TYPE	GPU	GPU MEM	CPU	HOST MEM	Command
248151	Ubuntu	0	Compute	0%	9312MiB	19%	1%	1421MiB /home/Ubuntu/miniconda3/envs/ai/bin/p
1259	root	0	Graphic	0%	10MiB	0%	0%	64MiB /usr/lib/xorg/Xorg vt1 -displayfd 3 -
1902	gdm	0	Graphic	0%	8MiB	0%	0%	142MiB /usr/bin/gnome-shell
27999	Ubuntu	0	Graphic	0%	6MiB	0%	0%	1227MiB handy

F2Setup
F6Sort
F9Kill
F10Quit
F12Save Config

9.6GB being used

A screenshot of a JupyterLab web interface. The browser address bar shows 'localhost:8888/notebooks/Untitled2.ipynb'. The JupyterLab header includes the logo, the name 'Untitled2', and 'Last Checkpoint: 2 minutes ago'. The top menu bar contains 'File', 'Edit', 'View', 'Run', 'Kernel', 'Settings', and 'Help'. Below the menu is a toolbar with icons for saving, opening, and running code. The main area shows a code cell with the text: 'Based on the image provided, the URL of the rental website is:' followed by a code comment '##https://www.drivenow.example/cars##'. The cursor is positioned at the end of the code comment. The bottom of the code cell shows the text: 'This URL is clearly visible in the browser's address bar at the top of the screen.'



```

Loading checkpoint shards: 100% 2/2 [00:01<00:00, 1.02it/s]

•[3]: image_path = "/home/Ubuntu/TransferData/images/math/p2.png"
image = load_image(image_path)
display(IPImage(filename=image_path))

prompt = """
Describe the image
"""

messages = [{
    "role": "user",
    "content": [
        {"type": "image", "image": image},
        {"type": "text", "text": prompt},
    ],
}]

# build chat text (no tokenization)
chat_text = processor.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)

# create both text+vision tensors
inputs = processor(text=[chat_text], images=[image], return_tensors="pt")

# ensure tensors are on the same device as the model
inputs = {k: v.to(model.device) for k, v in inputs.items()}

```

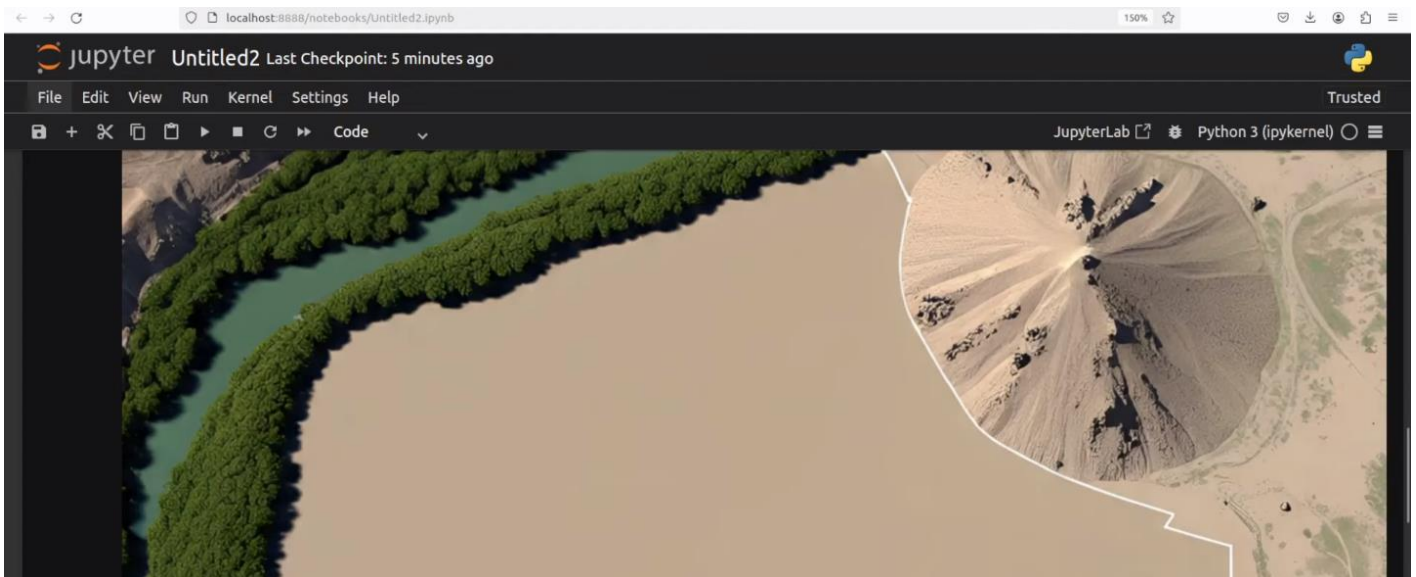
This image is a detailed schematic diagram of a deep learning model for **face retargeting**, specifically designed to transfer facial expressions (like eye and lip movements) from one person to another while preserving the appearance and motion of the target person. The model is likely used for applications like virtual try-ons, face swapping, or realistic avatar animation.

Here's a breakdown of the model's components and flow:

1. Input and Feature Extraction (Left Side)

- **Inputs:**
 - **I_s**: Source image (the person whose facial expression we want to transfer).
 - **I_d**: Target image (the

That's correct



```
Fetching 2 Files: 100% ██████████ 2/2 [00:00<00:00, 259.19it/s]
Loading checkpoint shards: 100% ██████████ 2/2 [00:01<00:00, 1.02it/s]

[5]: image_path = "/home/Ubuntu/TransferData/images/planner/satimage.jpeg"
      image = load_image(image_path)
      display(IPImage(filename=image_path))

      prompt = """
      can I build a house on this land?
      """

      messages = [{
        "role": "user",
        "content": [
          {"type": "image", "image": image},
          {"type": "text", "text": prompt},
        ],
      }]

      # build chat text (no tokenization)
      chat_text = processor.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)

      # create both text+vision tensors
      inputs = processor(text=[chat_text], images=[image], return_tensors="pt")

      # ensure tensors are on the same device as the model
```



```

JupyterLab Python 3 (ipykernel)
File Edit View Run Kernel Settings Help
Code
# build chat text (no tokenization)
chat_text = processor.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)

# create both text+vision tensors
inputs = processor(text=[chat_text], images=[image], return_tensors="pt")

# ensure tensors are on the same device as the model
inputs = {k: v.to(model.device) for k, v in inputs.items()}

# generate
generated_ids = model.generate(
    **inputs,
    max_new_tokens=128,
    do_sample=False,
    pad_token_id=processor.tokenizer.eos_token_id,
)

# trim prompt tokens before decoding (matches your pattern)
generated_ids_trimmed = [
    out_ids[len(in_ids):] for in_ids, out_ids in zip(inputs["input_ids"], generated_ids)
]

output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
print(output_text[0] if isinstance(output_text, list) else output_text)

```

JupyterLab Python 3 (ipykernel)

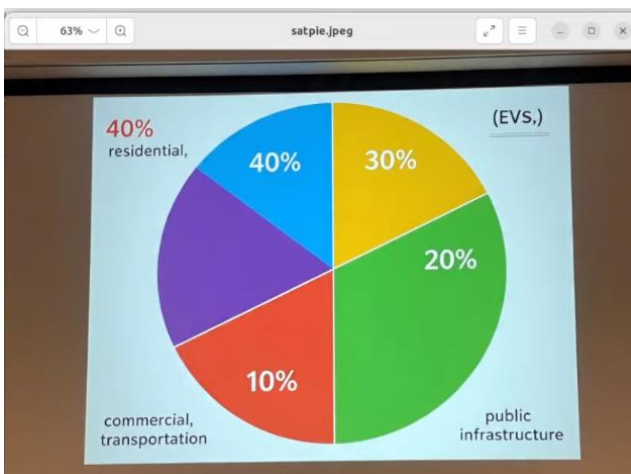


Based on the image provided, **it is highly unlikely you can build a house on this land**.

Here's why:

- The Land is a Large Body of Water:** The vast, flat, beige area is clearly a **lake or reservoir**. The white line drawn around it is likely a boundary or a digital rendering artifact, not a land parcel. You cannot build on water.
- The "Land" is Not Real Land:** The image shows a large, flat, water-filled area. The only "land" visible is the green forested area along the edge and the small, cone-shaped mountain.

Correct again



```
localhost:8888/notebooks/Untitled2.ipynb 150%
jupyter Untitled2 Last Checkpoint: 6 minutes ago
File Edit View Run Kernel Settings Help Trusted
+ % Code
processor = AutoProcessor.from_pretrained(
    "Owen/Owen3-VL-4B-Instruct",
    trust_remote_code=True,
)

Fetching 2 files: 100% 2/2 [00:00<00:00, 259.19it/s]
Loading checkpoint shards: 100% 2/2 [00:01<00:00, 1.02it/s]

•[5]: image_path = "/home/Ubuntu/TransferData/images/planner/satpie.jpeg"
image = load_image(image_path)
display(IPImage(filename=image_path))

prompt = """
How much is being spent on public infra?
"""

messages = [{
    "role": "user",
    "content": [
        {"type": "image", "image": image},
        {"type": "text", "text": prompt},
    ],
}]

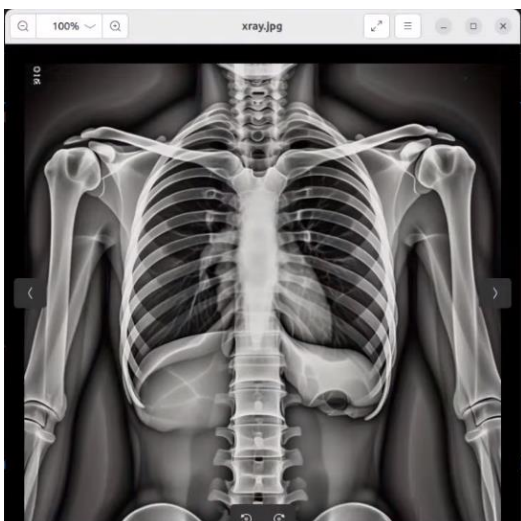
# build chat text (no tokenization)
chat_text = processor.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
```

```
localhost:8888/notebooks/Untitled2.ipynb 150%
jupyter Untitled2 Last Checkpoint: 7 minutes ago
File Edit View Run Kernel Settings Help Trusted
+ % Code
Based on the pie chart shown in the image, the amount being spent on public infrastructure is **20%**.

The chart breaks down the spending as follows:
- Residential: 40%
- Commercial, transportation: 10%
- Public infrastructure: 20%
- EVs: 30% (This is labeled as a separate category, not a spending category itself, but it's the largest portion at 30%)

Therefore, the percentage allocated to public infrastructure is 20%.
```

That's correct again



```
localhost:8888/notebooks/Untitled2.ipynb 150%
Jupyter Untitled2 Last Checkpoint: 8 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

In [6]: image_path = "/home/ubuntu/TransferData/images/med/xray.jpg"
image = load_image(image_path)
display(IPImage(filename=image_path))

prompt = """
Is this x-ray alright?
"""

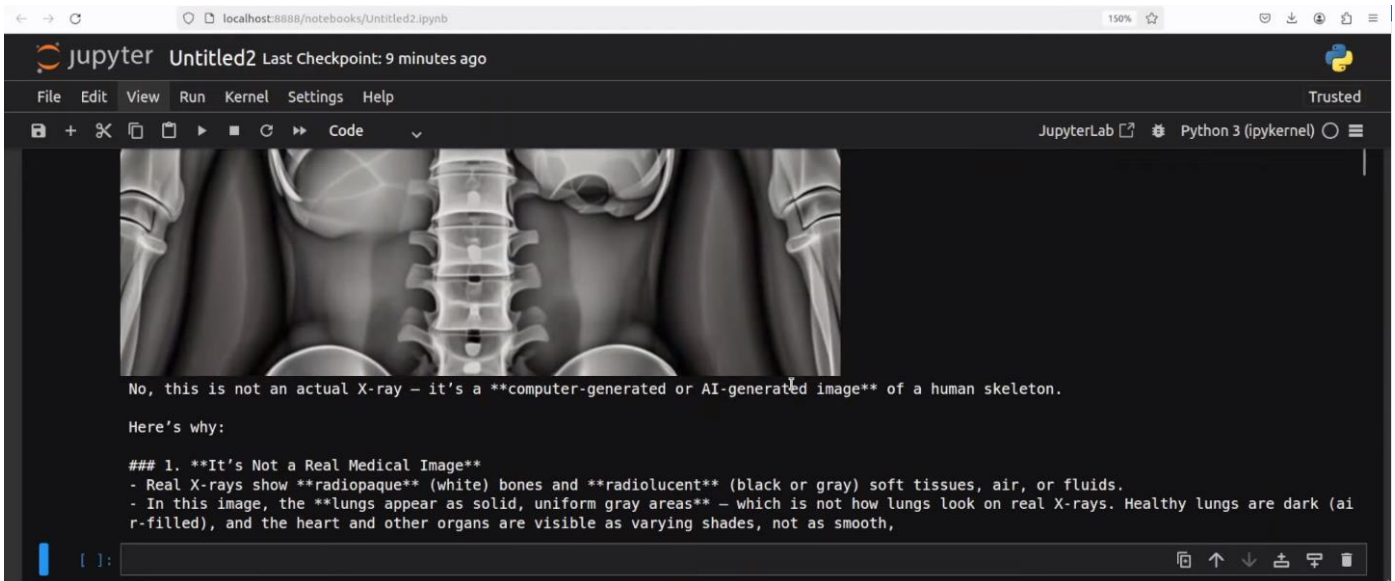
messages = [{
    "role": "user",
    "content": [
        {"type": "image", "image": image},
        {"type": "text", "text": prompt},
    ],
}]

# build chat text (no tokenization)
chat_text = processor.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)

# create both text+vision tensors
inputs = processor(text=[chat_text], images=[image], return_tensors="pt")

# ensure tensors are on the same device as the model
inputs = {k: v.to(model.device) for k, v in inputs.items()}

# generate
```



Correct again


```
# trim prompt tokens before decoding (matches your pattern)
generated_ids_trimmed = [
    out_ids[len(in_ids):] for in_ids, out_ids in zip(inputs["input_ids"], generated_ids)
]

output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
print(output_text[0] if isinstance(output_text, list) else output_text)
```

غالبًا ما تعتبر السعادة والرضا الأهداف النهائية في الحياة. في حين أن السعادة هي شعور عابر يمكن أن يتأثر بالعوامل الخارجية، فإن الرضا هو شعور أصق بالارتياح يأتي من الداخل. يتعلق بالامتنان لما لديك، وتنمية الامتنان، والعثور على الفرح في اللحظات البسيطة. تحقيق توازن بين السعي نحو تجارب جديدة وتقدير الحاضر يمكن أن يؤدي إلى حياة ممتدة. في النهاية، يوفر الرضا أساسًا مستقرًا للسعادة، مما يمكنك من التعامل مع تقلبات الحياة برشاقة ومرونة.

The extracted text is in Arabic.

Here is the text:

localhost:8888/notebooks/Untitled2.ipynb150%

jupyterUntitled2 Last Checkpoint: 13 minutes ago

FileEditViewRunKernelSettingsHelp

Python 3 (ipykernel)

generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False)

print(output_text[0] if isinstance(output_text, list) else output_text)

"Glück und Zufriedenheit werden oft als die ultimativen Ziele im Leben betrachtet. Während Glück ein flüchtiges Gefühl ist, das von äußeren Faktoren beeinflusst werden kann, ist Zufriedenheit ein tieferes Gefühl der Erfüllung, das von innen kommt. Es geht darum, das zu schätzen, was man hat, Dankbarkeit zu kultivieren und Freude in einfachen Momenten zu finden. Ein Gleichgewicht zwischen dem Streben nach neuen Erfahrungen und dem Wertschätzen der Gegenwart zu finden, kann zu einem erfüllten Leben führen. Letztlich bietet Zufriedenheit eine stabile Grundlage für Glück, die es ermöglicht, die Höhen und Tiefen des Lebens mit Anmut und Widerstandskraft zu meistern."

The extracted text is in German.

Text:

"Glück und Zufriedenheit werden oft als die ultimativen Ziele im Leben betrachtet. Während Glück ein flüchtiges Gefühl ist, das von äußeren Faktoren beeinflusst werden kann, ist Zufriedenheit ein tieferes Gefühl der Erfüllung, das von innen kommt. Es geht darum, das zu schätzen, was man hat, Dankbarkeit zu kultivieren und Freude in einfachen Momenten zu finden. Ein Gleichgewicht zwischen dem Streben nach neuen Erfahrungen und dem Wertschätzen der Gegenwart zu finden, kann zu einem erfüllten Leben führen. Letztlich bietet Zufriedenheit eine stabile Grundlage für Glück, die es ermöglicht, die Höhen und Tiefen des Lebens mit Anmut und Widerstandskraft zu meistern."